

CUSTOM CRICKET

CIS 293 – ADV TECH

TOPIC I PROJECT
PRESENTATION

ZACH KAHLER



CUSTOM CRICKET APP

- What is Cricket?
 - Cricket is a popular game played on a dart board. Using numbers 15 to 20 and the bullseye, players attempt to “close out” all the numbers by hitting them three times each. If you land a dart on a space you have closed out and another player has it “open” you score that number of points until ALL players have that number closed out.
- What is Custom Cricket?
 - This all started with a joke I made to my friend after he taught me Cricket.
 - “So... What if we used ALL THE NUMBERS?” – Zach K.
 - The rest is history.

APPLICATION ARCHITECTURE

- Dart Framework

- Dart is an open-source, object-oriented programming language developed by Google, designed for building fast, scalable, and portable applications, especially for web and mobile.



Dart

- Flutter UI

- Flutter is a UI toolkit and framework built on Dart, enabling developers to create natively compiled, cross-platform apps for mobile, web, and desktop from a single codebase.



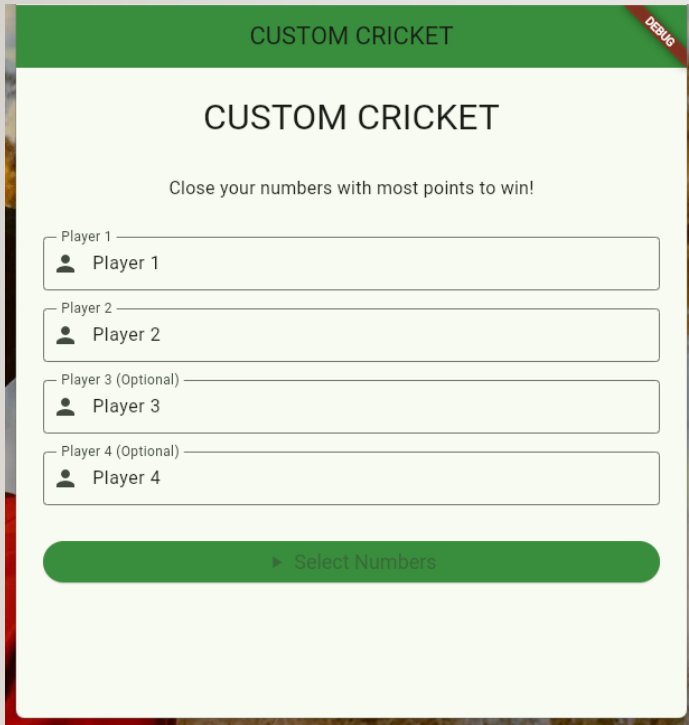
Flutter

APP ENTRY & MAIN WIDGET

```
lib > main.dart > DartsCricketScreen > createState
1  import 'package:flutter/material.dart';
2  import 'dart:math';
3
4  Run | Debug | Profile
5  void main() {
6    runApp(const MyApp());
7  }
8
9  class MyApp extends StatelessWidget {
10    const MyApp({super.key});
11
12    @override
13    Widget build(BuildContext context) {
14      return MaterialApp(
15        title: 'CUSTOM CRICKET',
16        theme: ThemeData(
17          colorScheme: ColorScheme.fromSeed(seedColor: Colors.green),
18          useMaterial3: true,
19        ), // ThemeData
20        home: const DartsCricketScreen(),
21      ); // MaterialApp
22    }
23  }
```

- This section initializes the app and sets up the main widget, MyApp, which uses Material Design and sets the home screen to DartsCricketScreen. (Lines 1-20)

PLAYER SET UP SCREEN



CUSTOM CRICKET

CUSTOM CRICKET

Close your numbers with most points to win!

Player 1

Player 1

Player 2

Player 2

Player 3 (Optional)

Player 3

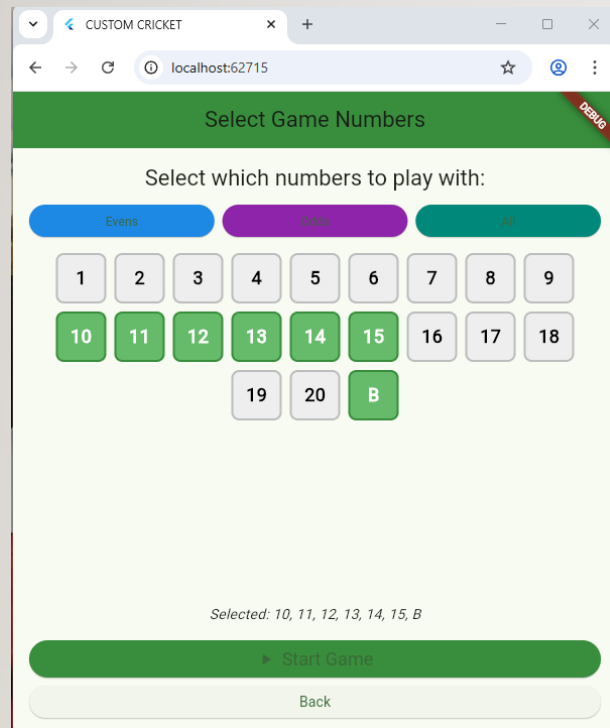
Player 4 (Optional)

Player 4

► Select Numbers

- The setup screen allows users to enter player names and start the game. It features text fields for up to four players and a button to proceed to number selection. (lines 61-120)

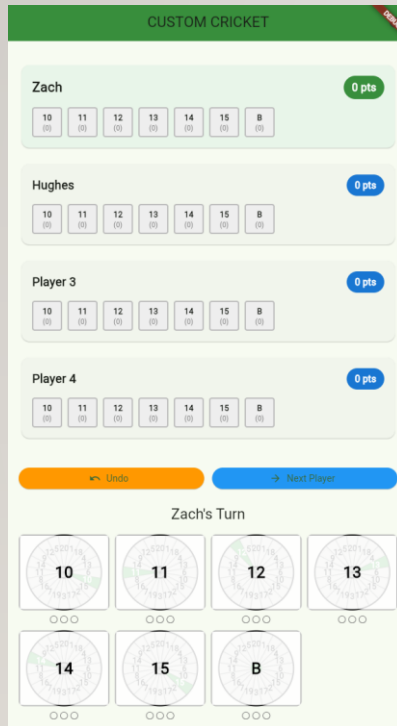
NUMBER SELECTION SCREEN



- Players can select which numbers to play with (evens, odds, all, or custom). The UI uses buttons and a grid for easy selection, making the game customizable. (Lines 181-300)

```
158 Widget _buildNumberSelectionScreen() {  
159   final allNumbers = ['1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12', '13', '14', '15', '16', '17', '18', '19', '20', '  
160   final evenNumbers = ['2', '4', '6', '8', '10', '12', '14', '16', '18', '20'];  
161   final oddNumbers = ['1', '3', '5', '7', '9', '11', '13', '15', '17', '19'];  
162  
163   return Scaffold(  
164     appBar: AppBar(  
165       title: const Text('Select Game Numbers'),  
166       centerTitle: true,  
167       backgroundColor: Colors.green[700],  
168     ),  
169     body: Center(  
170       child: Column(  
171         mainAxisAlignment: MainAxisAlignment.space-around,  
172         children: [  
173           Text('Select which numbers to play with:'),  
174           Row(  
175             mainAxisAlignment: MainAxisAlignment.space-around,  
176             children: [  
177               ElevatedButton('Evens', onPressed: () {}),  
178               ElevatedButton('Odds', onPressed: () {}),  
179               ElevatedButton('All', onPressed: () {}),  
180             ],  
181           ),  
182           GridView.builder(  
183             gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(  
184               crossAxisCount: 4,  
185             ),  
186             itemCount: 21,  
187             itemBuilder: (context, index) {  
188               if (index == 20) return ElevatedButton('B', onPressed: () {});  
189               return ElevatedButton(index.toString(), onPressed: () {});  
190             },  
191           ),  
192           Text('Selected: 10, 11, 12, 13, 14, 15, B'),  
193           ElevatedButton('Start Game', onPressed: () {}),  
194           ElevatedButton('Back', onPressed: () {}),  
195         ],  
196       ),  
197     ),  
198   );  
199 }
```

GAME SCREEN & GAMEPLAY



```
502 Widget _buildOpponentCard(CricketPlayer player) {
503   return Card(
504     child: Padding(
505       padding: const EdgeInsets.all(16.0),
506       child: Column(
507         crossAxisAlignment: CrossAxisAlignment.start,
508         children: [
509           Row(
510             mainAxisAlignment: MainAxisAlignment.spaceBetween,
511             children: [
512               Text(
513                 player.name,
514                 style: const TextStyle(
515                   fontSize: 18,
516                   fontWeight: FontWeight.bold,
517                 ), // TextStyle
518               ), // Text
519               Container(
520                 padding: const EdgeInsets.symmetric(
521                   horizontal: 12,
522                   vertical: 6,
523                 ), // EdgeInsets.symmetric
524                 decoration: BoxDecoration(
525                   color: Colors.blue[700],
526                   borderRadius: BorderRadius.circular(20),
527                 ), // BoxDecoration
528                 child: Text(
529                   '${player.score} pts',
530                   style: const TextStyle(
531                     color: Colors.white,
532                     fontWeight: FontWeight.bold,
533                   ), // TextStyle
534                 ), // Text
535               ), // Container
536             ],
537           ), // Row
538           const SizedBox(height: 12),
539           _buildNumbersStatus(player),
540         ],
541       ), // Column
542     ), // Padding
543   ); // Card
544 }
```

- The main game screen displays player cards, scores, and the current turn. Players interact with a dartboard grid to score, and the UI updates dynamically. (Lines 421 – 540)

DARTBOARD VISUALIZATION



- Custom painter code draws a dartboard, highlighting the selected number. This visual feedback enhances gameplay and user experience. (Lines 781-900)

```
// Draw bullseye circle
if (highlightNumber == 'B') {
  canvas.drawCircle(
    center,
    radius * 0.15,
    Paint()..color = Colors.green,
  );
} else {
  canvas.drawCircle(
    center,
    radius * 0.15,
    Paint()
      ..color = Colors.black
      ..style = PaintingStyle.stroke
      ..strokeWidth = 2,
  );
}
```

GAME LOGIC (MODELS)

```
// Models
class CricketGame {
  static const List<String> defaultNumbers = ['15', '16', '17', '18', '19', '20', 'B'];

  late List<CricketPlayer> players;
  late int currentPlayerIndex;
  late List<String> numbers;
  List<Move> moveHistory = [];

  CricketGame({
    List<String> playerNames = const ['Player 1', 'Player 2'],
    List<String>? customNumbers,
  }) {
    numbers = customNumbers ?? defaultNumbers;
    players = playerNames.map((name) => CricketPlayer(name, numbers)).toList();
    currentPlayerIndex = 0;
  }

  CricketPlayer get currentPlayer => players[currentPlayerIndex];

  CricketPlayer get otherPlayer => players[(currentPlayerIndex + 1) % players.length];
}
```

- This section initializes the app and sets up the main widget, MyApp, which uses Material Design and sets the home screen to DartsCricketScreen.

YOU WIN!

THANK YOU FOR WATCHING!

CUSTOM CRICKET

DEBUG

Zach

12 pts

10

11

12

13

14

15

B